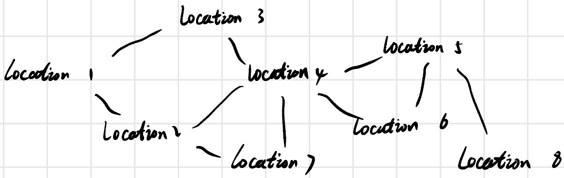
Imagine we have a map with many different locations:



Where the edges means the two locations are connected so we can commute between the two locations.

Question: Can we drive from location A to location B

Intuitive Solution: Breath first search or depth first search to check if A and B are connected.

problem: time complexity $O(n)$, n is the sample size (different locations)

If we ask the question many times, it becomes very slow.

Better solution: Disjoint Set Union

intuition: in order to check if location A and B are connected we don't actually need to find a path connecting A and B
we only need to find at least one location connecting to both A and B.

Speedup: time complexity $O(1)$

DSU intuitive explanation:

Initialization

Location 1   Location 2 , ... , Location n

Set 1: root → location 1   Set 2: root → location 2 ... , Set n: root → location n

Size   1   1 , ... ,   1

initialize the sets, each set with its root initialized as the location itself

Union

check all connected locations:

if Location A and location B is connected

find the root of the set where location A is in. Say root = i, set = S_i

find the root of the set where location B is in. Say root = j, set = S_j

if i = j, A and B are in the same set

if $i \neq j$, then they are in different set and we union the two sets

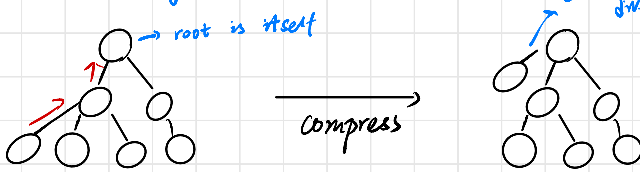
if $Size(S_i) \leq Size(S_j)$ → make j the root of i

we can make the i root of j. but this is more efficient

else make i the root of j

Find

Given a location, find the root of this set



Compression: in order to improve the efficiency for future search, we can compress the path by connecting the nodes in the path directly to the root.

Computation complexity

find: it can be shown that if we connect the set with smaller size to the larger set (as in the union step) and compress the path (as in the find step), perform find function once is mathematically $O(\alpha(n)) \approx O(1)$

inverse Ackermann function

Amortized time complexity
(average running time)

Union: union essentially performs two find, and then attach the smaller set to larger set, so $O(1)$

time complexity to build the DSU from scratch given n nodes:

$O(n+e)$

number of edges, perform e times union

initialize